



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Jiménez Elizalde Josué

N° de Cuenta: 320334489

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 04/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Como primer ejercicio se hizo de varios métodos, tanto el de los vértices repetidos y reutilizarlos, como asignarle un color en `vertices_letras[]`. Se hizo esto para cubrir más puntos y reforzar conocimientos.

Como primer pasos se hicieron métodos para la creación de cada letra en este caso es JOE, como los siguientes.

```
void CrearJ() {
    GLfloat J_vertices[] = {
        -0.70f, 0.90f, 0.0f,
        -0.65f, 0.90f, 0.0f,
        -0.70f, 0.10f, 0.0f,
        -0.65f, 0.10f, 0.0f,

        -0.95f, 0.15f, 0.0f,
        -0.65f, 0.15f, 0.0f,
        -0.95f, 0.10f, 0.0f,
        -0.65f, 0.10f, 0.0f,

        -0.95f, 0.20f, 0.0f,
        -0.90f, 0.20f, 0.0f,
        -0.95f, 0.10f, 0.0f,
        -0.90f, 0.10f, 0.0f,
    };

    unsigned int indices[] = {

        0,1,2,
        1,3,2,

        4,5,6,
        5,7,6,

        8,9,10,
        9,11,10
    };

    Mesh* J_letra = new Mesh();
    J_letra->CreateMesh(J_vertices, indices, 36, 18);
    meshList.push_back(J_letra);
};
```

Este es un ejemplo de la forma que se hizo, como los cubos y la pirámide se utilizó. Se reutilizaron vértices que se repetían, además da la oportunidad para que se pueda extender a una visualización 3D reutilizando los puntos.

La forma de utilización se resume en

```
shaderList[5].useShader();
uniformModel = shaderList[5].getModelLocation();
uniformProjection = shaderList[5].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.30f, 0.0f, -4.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.15f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
PARA QUE NO SEA TRANSPUESTA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[2]->RenderMesh();
```

Es un pequeño ejemplo de la implementación de la letra J, con las demás letras es semejante, solo que es cambiando el color con `shaderList[5]` cambiándolo a cada color que en este caso es verde. Además de utilizar la matriz de traslación para mover la letra y scale para hacerla más o menos grande.

ShaderList y su utilización:

```
#version 330
layout (location =0) in vec3 pos;
out vec4 vColor;
uniform mat4 model;
uniform mat4 projection;
void main()
{
    gl_Position=projection*model*vec4(pos,1.0f);
    vColor=vec4(0.0f, 0.0f, 1.0f,1.0f);
    //vColor=vec4(clamp(pos,0.0f,1.0f),1.0f);
}
```

Es un ejemplo de los demás shader, pero en este caso se modifíco para que sea de color Azul. El que esta comentado varia del color RGB entonces solo queremos el color Azul, así con los demás.

```
static const char* vShaderColorBlue = "shaders/shadercolorBlue.vert";
```

Se importa en el archivo main para su utilización.

```
Shader* shaderBlue = new Shader();
shaderBlue->CreateFromFiles(vShaderColorBlue, fShader);
shaderList.push_back(*shaderBlue);
```

Con esto se agrega en el arreglo para su utilización, simplemente lo utilizas como se menciono anteriormente.

En forma diferente de poner las letras se tiene:

```
GLfloat vertices_letras[] = {
    //X           Y           Z           R           G

    //J

    -0.70f, 0.90f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.65f, 0.90f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.70f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,

    -0.65f, 0.90f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.65f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.70f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,

    -0.95f, 0.15f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.65f, 0.15f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.95f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,

    -0.65f, 0.15f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.65f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.95f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,

    -0.90f, 0.20f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.90f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.95f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,

    -0.95f, 0.20f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.90f, 0.20f, 0.0f,      1.0f, 0.0f, 0.0f,
    -0.95f, 0.10f, 0.0f,      1.0f, 0.0f, 0.0f,

    //O

    //Parte superior
    -0.60f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.30f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.60f, 0.85f, 0.0f,      0.0f, 0.0f, 1.0f,

    -0.30f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.30f, 0.85f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.60f, 0.85f, 0.0f,      0.0f, 0.0f, 1.0f,

    //Parte inferior
    -0.60f, 0.15f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.30f, 0.15f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.60f, 0.10f, 0.0f,      0.0f, 0.0f, 1.0f,

    -0.30f, 0.15f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.30f, 0.10f, 0.0f,      0.0f, 0.0f, 1.0f,
    -0.60f, 0.10f, 0.0f,      0.0f, 0.0f, 1.0f,

    //Lateral izquierdo
    -0.60f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
```

```

-0.55f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
-0.60f, 0.10f, 0.0f,    0.0f, 0.0f, 1.0f,

-0.55f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
-0.55f, 0.10f, 0.0f,    0.0f, 0.0f, 1.0f,
-0.60f, 0.10f, 0.0f,    0.0f, 0.0f, 1.0f,

//Lateral derecho
-0.35f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
-0.30f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
-0.35f, 0.10f, 0.0f,    0.0f, 0.0f, 1.0f,

-0.30f, 0.9f, 0.0f,      0.0f, 0.0f, 1.0f,
-0.30f, 0.10f, 0.0f,    0.0f, 0.0f, 1.0f,
-0.35f, 0.10f, 0.0f,    0.0f, 0.0f, 1.0f,

//E

//Palito vertical
-0.20f, 0.9f, 0.0f,      0.0f, 1.0f, 0.0f,
-0.15f, 0.9f, 0.0f,      0.0f, 1.0f, 0.0f,
-0.20f, 0.1f, 0.0f,      0.0f, 1.0f, 0.0f,

-0.15f, 0.9f, 0.0f,      0.0f, 1.0f, 0.0f,
-0.15f, 0.1f, 0.0f,      0.0f, 1.0f, 0.0f,
-0.20f, 0.1f, 0.0f,      0.0f, 1.0f, 0.0f,

//Línea superior
-0.20f, 0.9f, 0.0f,      0.0f, 1.0f, 0.0f,
0.05f, 0.9f, 0.0f,      0.0f, 1.0f, 0.0f,
-0.20f, 0.85f, 0.0f,    0.0f, 1.0f, 0.0f,

0.05f, 0.9f, 0.0f,      0.0f, 1.0f, 0.0f,
0.05f, 0.85f, 0.0f,      0.0f, 1.0f, 0.0f,
-0.20f, 0.85f, 0.0f,    0.0f, 1.0f, 0.0f,

//Línea del medio
-0.20f, 0.55f, 0.0f,    0.0f, 1.0f, 0.0f,
0.00f, 0.55f, 0.0f,    0.0f, 1.0f, 0.0f,
-0.20f, 0.50f, 0.0f,    0.0f, 1.0f, 0.0f,

0.00f, 0.55f, 0.0f,    0.0f, 1.0f, 0.0f,
0.00f, 0.50f, 0.0f,    0.0f, 1.0f, 0.0f,
-0.20f, 0.50f, 0.0f,    0.0f, 1.0f, 0.0f,

//Línea inferior
-0.20f, 0.15f, 0.0f,    0.0f, 1.0f, 0.0f,
0.05f, 0.15f, 0.0f,    0.0f, 1.0f, 0.0f,
-0.20f, 0.10f, 0.0f,    0.0f, 1.0f, 0.0f,

0.05f, 0.15f, 0.0f,    0.0f, 1.0f, 0.0f,
0.05f, 0.10f, 0.0f,    0.0f, 1.0f, 0.0f,
-0.20f, 0.10f, 0.0f,    0.0f, 1.0f, 0.0f,

};
MeshColor* letras = new MeshColor();
letras->CreateMeshColor(vertices_letras, 396);
meshColorList.push_back(letras);

```

Cada punto con un color elegido como se muestra en los comentarios, que representa cada valor de este arreglo. Después de eso se agrega al arreglo de meshColorList para su utilización futura.

```
shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.1f, -4.0f));
model = glm::scale(model, glm::vec3(0.7f, 1.0f, 0.30f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshColorList[0]->RenderMeshColor();
```

Esta es la forma de utilizarlo, como se muestra es más sencilla su utilización pero su creación es muy grande.

En el siguiente ejercicio, solamente se utilizaron los cubos y pirámides ya creados como las letras de la primera forma, su utilización fue similar solo cambiando el shader a utilización del objeto.

//Cubos rojos

```
shaderList[2].useShader();
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.5f, -4.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.7f, 0.15f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
PARA QUE NO SEA TRANSPUESTA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();
```

//Cubos cafes

```
shaderList[4].useShader();
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.75f, -0.85f, -4.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.30f, 0.30f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
PARA QUE NO SEA TRANSPUESTA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();
```

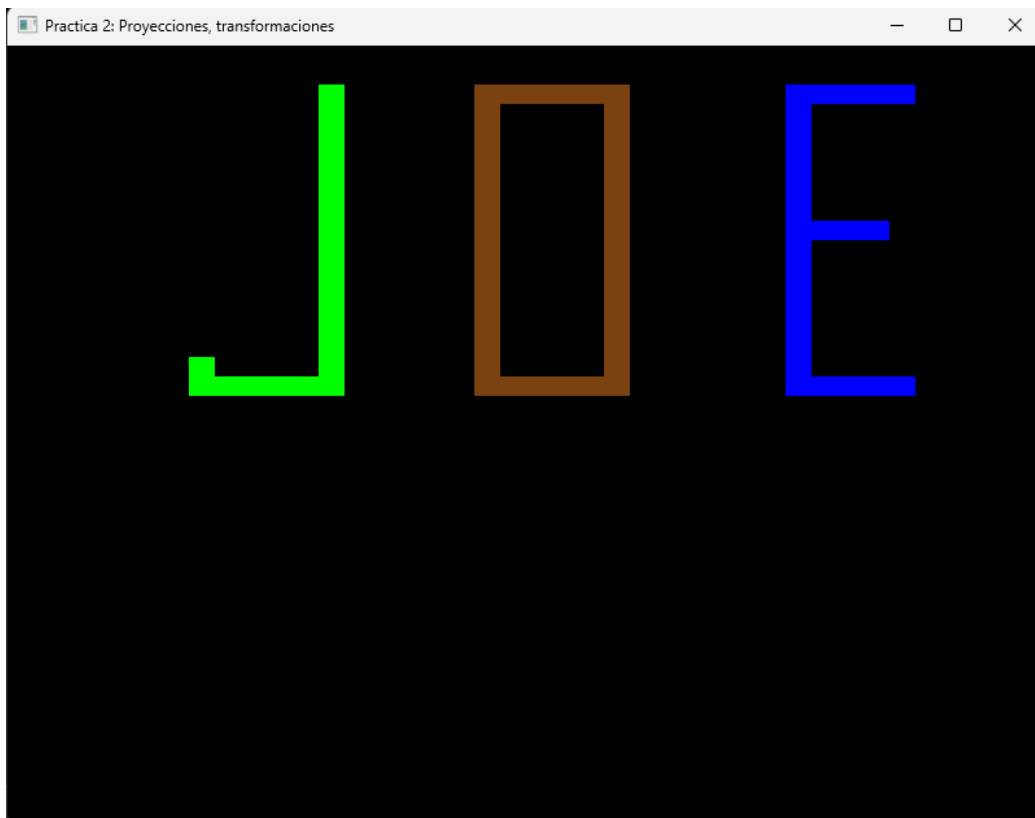
Como se muestra en la pequeña parte del código se tiene dos cubos de distintos colores para mostrar su implementación de forma sencilla y no todo el código. Lo

que cambia con las pirámides en `meshList[1]->RenderMesh()`; que es el primer elemento, por lo que para imprimir la pirámide es `meshList[0]->RenderMesh()`;

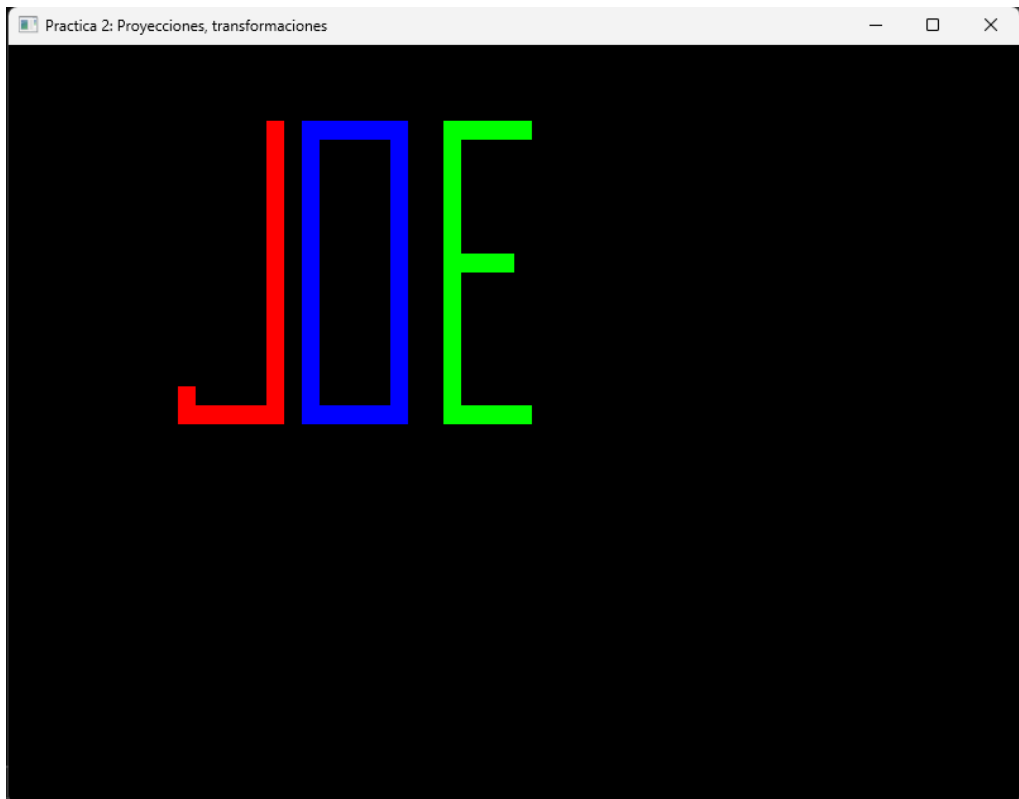
Captura de pantalla de la resolución:

Letras.

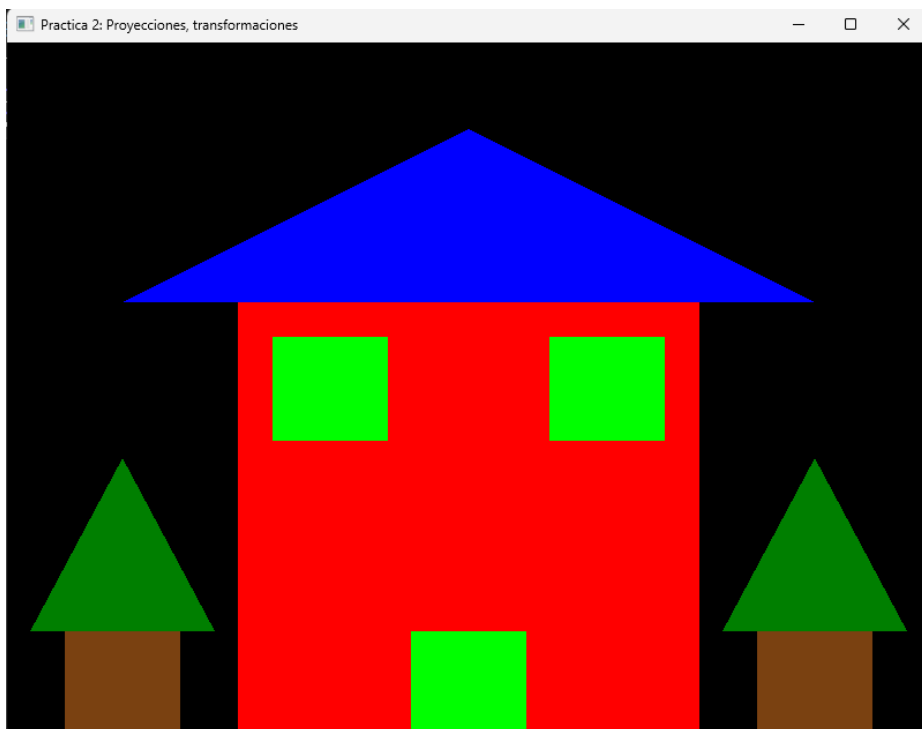
Letras de forma de reutilización de vértices.



Letras con color ya asignado:



Casa con cubos y pirámides.



Por lo que se muestra la diferencia de imprimir letras es por los colores y tamaños para identificarlos de manera rápida.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Los problemas que se presentaron fue colocar los vértices de forma correcta, además de los colores ya que a la hora de poner colores con `vertices_letras[]` me di cuenta de que si pones distintos colores en los puntos, el triangulo que se va a formar es multicolor ya que cada vértice se pinta del color asignado.

Fuera de eso no se presentó ningún problema.

3.- Conclusión:

La complejidad de los ejercicios es adecuada a lo visto en la práctica, además se obtuvieron esos conocimientos a la hora de elaborar el previo. Entonces no se tuvieron complicaciones en la resolución de la práctica, además de colocar bien los vértices.

Bibliografía en formato APA

LearnOpenGL - *Shaders*. (s. f.). <https://learnopengl.com/Getting-started/Shaders>